Wipro NGA .NET Cohort

Daily Coding Assignment

SOLID Principles and Design Patterns in .NET

Name: Diksha Chaudhary

Technology: .NET

Assignment: SOLID Principles and Design Patterns

Part 1: Applying SOLID Principles

Objective

The objective of this assignment was to refactor an existing reporting system so that it follows the SOLID principles. The refactored solution focuses on separation of concerns, loose coupling, extensibility, and testability.

Single Responsibility Principle

Initially, a single class was responsible for generating the report as well as saving it to a file. This violated the Single Responsibility Principle.

The functionality was divided into two separate classes:

- ReportGenerator – responsible for generating the report content
- ReportSaver – responsible for saving the generated report

This ensures that each class has only one reason to change and improves maintainability.

Open/Closed Principle

To make the system open for extension and closed for modification, a common interface for formatting was introduced.

An interface named IReportFormatter was created and multiple implementations were provided:

- PdfFormatter
- ExcelFormatter

With this design, new formats can be added without modifying the existing code.

Liskov Substitution Principle

A base class named Report was created and extended by specialized report types such as SalesReport.

The derived classes properly override the base behavior and can be used wherever the base class is expected without affecting program correctness.

This confirms that the system follows the Liskov Substitution Principle.

Interface Segregation Principle

Instead of using a large interface with multiple unrelated methods, smaller and more specific interfaces were used.

This ensures that a class only implements the operations that are relevant to it and is not forced to implement unused methods.

This improves code clarity and reduces unnecessary dependencies.

Dependency Inversion Principle

The high-level module ReportService was designed to depend on abstractions rather than concrete implementations.

Dependencies such as:

- ReportGenerator
- IReportFormatter
- ReportSaver

were injected through the constructor.

This makes the system loosely coupled and easily testable.

Unit Testing

Unit tests were written to verify the functionality of the refactored system. The tests ensure that:

- Reports are generated and saved successfully
- Different formatter implementations work without modifying the service

This confirms that the system is extensible and follows SOLID principles.

Screenshot to be added:

Test methods and test execution result showing all tests passed.

```
(base) dikshachaudhary@Dikshas-MacBook-Air WiproSolidAssignment % dotnet tes
t
Restore complete (0.6s)
  SolidPrinciplesDemo net10.0 succeeded (0.2s) → SolidPrinciplesDemo/bin/Debug/net10.0/SolidPrinciplesDemo.dll
  SolidPrinciplesDemo.Tests net10.0 succeeded (0.3s) → SolidPrinciplesDemo.Tests/bin/Debug/net10.0/SolidPrinciplesDemo.Tests.dll
Test Parallelization enabled for /Users/dikshachaudhary/WiproSolidAssignment/SolidPrinciplesDemo.Tests/bin/Debug/net10.0/SolidPrinciplesDemo.Tests.dll (Workers: 8, Scope: MethodLevel)
  SolidPrinciplesDemo.Tests test net10.0 succeeded (1.6s)

Test summary: total: 2, failed: 0, succeeded: 2, skipped: 0, duration: 1.5s
Build succeeded in 2.9s
```

Part 2: Design Pattern Implementation

Selected Design Pattern: Factory Pattern

The Factory Pattern was implemented to create document objects without exposing the instantiation logic to the client.

An interface named IDocument was created and implemented by:

- PdfDocument
- WordDocument

A DocumentFactory class was introduced to return the appropriate document type based on the input parameter.

This approach promotes loose coupling and makes the system easier to extend.

Implementation Details

The factory method returns objects through the interface rather than concrete classes. This allows the client code to work with abstractions.

If a new document type needs to be added in the future, it can be introduced without modifying the client logic.

Unit Testing of Factory Pattern

Unit tests were written to verify that:

- The factory returns the correct document type
- The returned object is of the expected implementation

This ensures that the pattern is working correctly.

Screenshot to be added:

Factory test methods and successful test results.

```
(base) dikshachaudhary@Dikshas-MacBook-Air WiproSolidAssignment % dotnet tes
t
Restore complete (0.6s)
  SolidPrinciplesDemo net10.0 succeeded (0.3s) → SolidPrinciplesDemo/bin/Debug/net10.0/SolidPrinciplesDemo.dl
l
  SolidPrinciplesDemo.Tests net10.0 succeeded (0.3s) → SolidPrinciplesDemo.Tests/bin/Debug/net10.0/SolidPrinc
iplesDemo.Tests.dll
Test Parallelization enabled for /Users/dikshachaudhary/WiproSolidAssignment/SolidPrinciplesDemo.Tests/bin/De
bug/net10.0/SolidPrinciplesDemo.Tests.dll (Workers: 8, Scope: MethodLevel)
  SolidPrinciplesDemo.Tests test net10.0 succeeded (1.5s)

Test summary: total: 4, failed: 0, succeeded: 4, skipped: 0, duration: 1.5s
Build succeeded in 2.9s
```

Outcome

After refactoring, the application:

- Follows all five SOLID principles
- Is loosely coupled and easier to maintain
- Can be extended without modifying existing code
- Is fully testable through unit tests

The implementation of the Factory Pattern demonstrates how object creation can be managed efficiently while maintaining flexibility and scalability.

Conclusion

This assignment provided practical exposure to applying SOLID principles in a real-world scenario and implementing a commonly used design pattern.

The refactored architecture improves readability, testability, and extensibility. The use of abstractions and dependency injection ensures that the system remains adaptable to future requirements.